JAVA

DEFINICIÓN DE CLASES

Declaración de clase:

```
[Modificadores] class Nombre [extends Clase] [implements Interfaz, ...] {
        // Atributos
        // Métodos
}
```

Java no impone ningún orden particular en el que hay que definir los elementos.

Modificadores:

- public: Permite que la clase sea accesible desde otro paquete (no confundir con los especificadores de visibilidad de atributos).
- abstract: Define clases que no pueden instanciarse.
- final: Define clases que no pueden extenderse con subclases.
- Cláusula extends: Define la superclase de la clase actual (por defecto Object).
- Cláusula implements: Define los interfaces que implementa la clase.

CREACIÓN DE OBJETOS

Se crean usando el operador new junto con el constructor de la clase de la que queremos crear un objeto.

ESTADO DE OBJETOS

Declaración de atributo

```
[Especificador de acceso][Modificadores] tipo nombreAtributo [= valor inicial];
```

- **Especificadores de acceso**: Definen desde dónde podemos acceder a ese atributo.
    - **Público** (public): Los atributos son visibles para todas las clases.
    - **Protegido** (protected): Permite que el atributo sea visible a las subclases de la clase original. También permite el acceso a las clases que se encuentren en el mismo paquete (a diferencia de otros lenguajes de programación).
    - **Paquete** (no se indica especificador): Es el que se aplica si no se especifica ningún especificador. Permite que el atributo sea visible a todas las clases que se sitúan en el mismo paquete que la clase original.
    - **Privado** (private): Los atributos sólo son visibles dentro de la propia clase.

Visibilidad	Misma clase	Subclase (mismo paquete)	Otra clase (mismo paquete)	Subclase (otro paquete)	Otra clase (otro paquete)
public	SÍ	SÍ	SÍ	SÍ	SÍ
protected	SÍ	SÍ	SÍ	SÍ	NO
[package]	SÍ	SÍ	SÍ	NO	NO
private	SÍ	NO	NO	NO	NO

- **Modificadores**: Permiten definir atributos de clase y atributos constantes.

@informaticapau

- static: Los atributos pertenecen a la clase y no a una instancia en particular. Pueden ser modificados sin que exista una instancia creada de la clase. Son compartidos por todas las instancias de la clase. Se acceden anteponiendo el nombre de la clase al nombre del atributo.
- final: Atributos constantes. Una vez se les ha asignado un valor no es posible cambiarlo.

- **Valor inicial**: Todos los atributos se inicializan automáticamente: 0 si es numérico, false si es booleano y null si es un objeto. En la definición se puede definir, si se considera necesario, un valor inicial distinto a dicho atributo.

COMPORTAMIENTO DE OBJETOS

Declaración de método

[Especificador de acceso][Modificadores] tipoRetorno nombreMetodo ([parámetro1, parámetro2, ...]) [throws ListaExcepciones] {}

- **Especificadores de Acceso**: Iguales que en los atributos.
- **Modificadores**:
  - static: Métodos de clase.
  - abstract: Métodos no definidos destinados a ser implementados por una subclase.
  - final: Evita que un método sea sobrescrito Si una clase es final todos sus métodos son implícitamente final.
  - native: Escritos en otro lenguaje (actualmente C y C++).
  - synchronized: Usado en la programación multihilo, un método sincronizado no puede ser ejecutado simultáneamente por dos hilos de ejecución.
- **Clausula** throws: Excepciones comprobadas que puede generar el método.

Paso de parámetros

Todos los parámetros en Java se pasan por valor.

Tipos básicos:

Los parámetros actuales (int, float, etc.) se copian en los parámetros formales.

Objetos:

Los objetos se implementan a través de punteros. Cuando pasamos un objeto como parámetro lo que se pasa por valor es su puntero. En la practica el objeto en sí (que reside en el montículo), lo estamos pasando por referencia.

Ensombrecimiento

Los parámetros ensombrecen el nombre de los atributos en caso de tener el mismo nombre. Esto quiere decir que, dentro de un método, en caso de ambigüedad, Java entenderá que nos estamos refiriendo al elemento más cercano (el parámetro). El uso del puntero this que representa al objeto actual puede romper esta ambigüedad.

Tipos de Retorno

Los métodos de Java devuelven un tipo de retorno que puede ser: un objeto, un tipo básico o un valor void (que significa que el método no devuelve nada). Para devolver un valor dentro del método se usa la palabra clave return que además termina la ejecución de dicho método (no importa si estamos dentro de un bucle).

@informaticapau

Los métodos sólo pueden devolver un elemento (y los parámetros se pasan por valor). Para devolver varios elementos lo habitual es agruparlos en un objeto.

Métodos de clase

Métodos estáticos y el puntero this

Al no pertenecer a una instancia no pueden acceder a atributos y métodos de instancia ya que no existe el puntero explícito this (debería crearse un objeto para poder acceder).

Forma de uso:

No se corresponden exactamente con la ortodoxia de la programación orientada a objetos.

Una llamada a un método de clase sería como un mensaje pasado a la clase, no a una instancia concreta de la clase. Son útiles cuando el método no necesita acceder al estado del objeto para realizar las operaciones (toda la información está en los parámetros).

Métodos constructores

- Deben llevar el mismo nombre de la Clase.
- No deben devolver ningún valor, ni siquiera void.
- Se llaman usando el operador new.

Hay que distinguir dos fases en la creación de un objeto:

- Creación propiamente dicha a través del operador new.
- Inicialización que se hace a través del propio método constructor una vez creado el objeto.

Por dicho motivo los constructores pueden acceder al puntero this que representa al objeto actual.

Si no se especifica un constructor en una clase Java asume que existe un constructor por defecto (sin parámetros). En el momento en el que se especifica explícitamente un constructor, el constructor por defecto desaparece (si queremos un constructor sin parámetro habrá que hacerlo explicito).

Encadenamiento de constructores

Un método puede tener varios constructores siempre que su número o tipo de parámetros sea distinto. Es habitual que los constructores se llamen entre si usando la palabra clave this.

Métodos destructores y recolección de basura:

En Java no existen, se usa la “recolección de basura”, lo que libera al programador de la responsabilidad del manejo de la memoria.

Otros métodos:

- Métodos **toString**: Devuelve una representación en String de un objeto. La implementación por defecto imprime nombreClase + @ + hashCode (no muy útil), si se quiere usar es mejor redefinirlo. Java detecta cuando necesitamos una representación en String de un objeto y llama automáticamente a este código.
- Método **main**: Cualquier clase puede tener un método main. Ejecutar un programa en Java significa ejecutar el método main de una clase particular de ese programa.

```
public static void main(String[] args) {}
```

@informaticapau

IDENTIDAD DE OBJETOS

En Java el OID (object identifier) de un objeto es la dirección de memoria en la cual reside el objeto.

Implementación de OIDs:

Una variable de tipo objeto en Java es en realidad un puntero al objeto que reside en el montículo.

Comparación de objetos

- Comparación de **identidad**: mediante el operador ==.
- Comparación de **igualdad**: mediante el método equals.

La comparación de identidad es fácil de realizar por el sistema: si los OIDs son distintos los objetos no son idénticos.

La comparación de igualdad depende del contexto. Es la propia clase la que tiene que definir el método equals para que indique qué atributos entran dentro de la comparación de igualdad y cuáles no.

Cuando una clase tiene la noción de igualdad lógica que difiere de la mera identidad de objetos, entonces dicha clase debe redefinir el método equals.

Contrato del método equals de Object

- **Reflexividad**: x.equals(x) debe devolver true.
- **Simetría**: x.equals(y) debe devolver lo mismo que y.equals(x).
- **Transitividad**: Si x.equals(y) devuelve true y y.equals(z) devuelve true entonces x.equals(z) debe devolver true.
- **Consistencia**: Si no se modifica el estado de los objetos que se usa para la comparación repetidas llamadas a x.equals(y) deberían responder siempre lo mismo.
- **Uso de nulos**: x.equals(null) debe devolver false para cualquier valor x no nulo.

```
@Override
public boolean equals(Object obj) {
    if (this == obj) {return true;}
    if (obj == null) {return false;}
    if (getClass() != obj.getClass()) {return false;}
    Clase other = (Clase) obj;
    return this.valor == other.valor;

}
```

- La firma de equals obliga a pasar un Object (superclase común) por parámetro.
- Si los objetos son idénticos entonces son iguales.
- Si obj es nulo la comparación es falsa.
- Si la clase de obj no es igual a la clase actual la comparación es falsa.

Precauciones a la hora de redefinir el equals

No substituir tipo del objeto que pasamos por parámetro, tiene que ser Object. Si usamos la anotación @Override (recomendado) el compilador sí avisará de que no estamos haciendo una sobrescritura.

@informaticapau

Contrato general del método hashCode de Object

El método hashCode debe consistentemente devolver el mismo valor entero, siempre y cuando la información usada en las comparaciones de igualdad no es modificada. No tiene por qué ser consistente en distintas ejecuciones de la misma aplicación.

Si dos objetos son iguales (de acuerdo con el método equals), entonces llamar al método hashCode sobre ambos objetos debe producir el mismo resultado. Si no, pueden tener el mismo valor de hashCode, pero producir valores hash distintos para objetos distintos mejora el rendimiento de la aplicación.

Un método sencillo para obtener un hashCode razonablemente eficiente consiste en obtener una representación entera de cada campo involucrado en el equals y sumarlo y multiplicarlo sucesivamente por dos números primos arbitrarios.

Siempre es necesario comprobar que se están usando los mismos campos para implementar tanto el equals como el hashCode.

equals y hashCode de Object

- El equals por defecto hace una comparación de identidad.
- El hashCode por defecto devuelve una representación entera de la dirección de memoria del objeto (cada objeto tiene un valor hash único).

TIPOS ENUMERADOS

Java inicialmente no incluía tipos enumerados, por lo que se definían como constantes enteras.

El patrón "Enumerado de Tipos Seguros" (*TypeSafe Enum*) aumentaba la seguridad usando constructores privados. El *TypeSafe Enum* podía llegar a complicarse bastante si añadíamos cosas como conversión a String, relaciones de orden entre los valores, etc. Todas muy deseables para trabajar con ellos.

Se utiliza enum para definir una clase cuyo constructor es privado, pero en la que existen constantes de clase (static y final). Al ser clases los enumerados aceptan que se le definan atributos y métodos. También podemos crear métodos constructores. Los valores enumerados se crearán usando estos constructores.

Ventajas de los tipos enumerados:

- **Igualdad**: Pueden hacerse comparaciones de igualdad usando == y equals.
- **Orden**: Implementan el interfaz Comparable por lo que se pueden comparar usando el método "compareTo". El método "ordinal" devuelve el valor ordinal de cada enumerado. El método "values" devuelve un array de enumerados que puede usarse en un bucle for.
- **Enumerados y Strings**: Sobrescriben el método "toString". Tienen un método "valueOf" que hace el efecto contrario.

@informaticapau

MODULARIDAD

La modularidad se organiza en Java de forma lógica en clases y paquetes y de forma física mediante ficheros y directorios.

- **Clases**: Encapsulan los atributos y métodos de un tipo de objetos en un solo compartimento. Ocultan, mediante los especificadores de acceso, los elementos internos que no se pretende publicar al exterior. Esta protección es altamente configurable al existir varios niveles de acceso.
- **Ficheros Java**: Actúan como unidades físicas de compilación. Dentro de un fichero .java pueden residir varias clases con las siguientes restricciones:
    - Solo puede haber una clase publica por fichero.
    - El nombre del fichero debe ser el mismo que el de la clase pública.
    - Si no existe ninguna clase publica tampoco existe ninguna restricción con respecto al nombre del fichero.

  La compilación de un fichero .java genera tantos ficheros .class (*bytecodes*) como clases existen en dicho fichero. Por simplicidad muchas veces se recomienda que un fichero contenga solo una única clase.
- **Paquetes Java**: Son unidades lógicas de agrupación de clases. Las clases que no son públicas solo son visibles dentro del propio paquete.

Definición de paquetes

- Se utiliza la directiva package al principio de cada fichero.
- Todas las clases definidas en ese fichero se consideran que pertenecen a ese paquete.
- En caso de que no se especifique se considera que todas las clases se encuentran en el paquete por defecto (aunque se recomienda no usarlo).
- Los nombres de los paquetes suelen ir en minúsculas (sin notación "*Camel Case*").

El nombre de una clase es siempre el nombre de su paquete + un punto + el propio nombre de la clase: paquete.Clase

Objetivos de los paquetes

- **Diseñar un dispositivo de modularidad superior a las clases**: Dentro de un paquete existen clases públicas (visibles al exterior) y clases no públicas (destinadas a ser usadas dentro del paquete). Se usa el modificador de visibilidad de las clases para establecer su carácter.
- **Agrupar clases con funcionalidades similares**: Aunque nuestras clases sean todas públicas es común agrupar clases relacionadas en paquetes para facilitar su organización.
- **Organizar físicamente los ficheros fuente**: Java equipara los nombres de paquetes con los directorios en disco, lo que permite tener organizados los ficheros fuente.
- **Prevenir conflictos de nombre**: Los paquetes permiten minimizar los conflictos de nombre en clases con nombres iguales, ya que el nombre de la clase siempre lleva antepuesto el nombre del paquete.

Jerarquías de paquetes

Los nombres de los paquetes pueden tener varios niveles separados por puntos. Aunque el nombre del paquete puede tener varios niveles no existe el concepto de "subpaquete" ni relaciones jerárquicas entre los paquetes. Se trata de una relación jerárquica de nombres por motivos de organización y claridad.

@informaticapau

¿Cómo prevenimos los conflictos con los nombres de los paquetes?

Para evitarlos existe el convenio de utilizar como prefijo a los nombres de los paquetes los nombres invertidos del dominio de Internet de la empresa que los ha desarrollado (que son únicos)

Importación de paquetes

La sentencia import especifica que los contenidos públicos del paquete especificado entran en el espacio de nombres del fichero de origen. De esta forma podemos usar el nombre de la clase sin anteponer el nombre del paquete. La importación se especifica justo después de la declaración package.

Formas de importación:

- Importar **una clase concreta**: import paquete.Clase
- Importar **todas las clases públicas** de un paquete: import paquete.*

Se recomienda usar la primera porque es más clara de cara al desarrollador (queda perfectamente detallado qué clases se están usando y cuáles no).

La importación de clases concretas crea listas largas al principio de las clases, pero los IDEs modernos permiten mantenerlas de forma sencilla (sugieren los paquetes que faltan, avisan de importaciones no usadas, etc.).

- No es necesario incluir la sentencia import para dar privilegios de acceso de un paquete a otro.
- Siempre podremos tener acceso a una clase usando su nombre cualificado completo paquete.Clase sin necesidad de hacer el import.
- Solo se requiere que la clase sea pública y esté accesible en el *CLASSPATH*.
- Por comodidad las clases pertenecientes al paquete java.lang (Object, etc.) no es necesario importarlas o usar su nombre completo para poder usarlas.

JERARQUÍA

Herencia

En Java la herencia se especifica usando la cláusula extends al principio de la definición de una clase.

Clase raíz

Es la estructura propia de Java en la que la clase raíz es la clase Object. Si no se incluye la cláusula extends en la definición de una clase se sobreentiende que está heredando directamente de Object.

Herencia y construcciones

La palabra clave super se usa para llamar a los constructores de la superclase (como this se usaba para llamar a otro constructor de la misma clase). No se puede usar super y this en un mismo constructor.

Clases abstractas

Métodos abstractos

Se definen en anteponiendo la palabra clave abstract.

Interfaces

Se definen con la palabra clave interface y no class.

@informaticapau

Declaración de un interfaz

```
[public] interface Nombre [extends Superinterfaz, ...]{
        // Atributos de clase
        // Métodos abstractos
}
```

Implementación de interfaces

La implementación se hace a través de la palabra clave implements.

Una clase que implementa un interfaz está obligada a darle una implementación a todos los métodos del interfaz.

POLIMORFISMO

Sobrescritura

En Java, al contrario que en otros lenguajes, no es necesario indicar explícitamente que se está haciendo una sobrescritura, simplemente con reproducir la firma del método en una subclase es suficiente.

- Problema: Cualquier fallo en la escritura del método de la subclase puede invalidar la sobrescritura. El compilador aceptará el código y el problema pasará inadvertido y puede que sea difícil de detectar.
- Solución: La anotación @Override situada antes de la sobrescritura de un método fuerza al compilador a fallar si la sobrescritura no es correcta. Es una anotación opcional, pero se recomienda su uso.

Sobrescritura de refinamiento.

Los métodos de la superclase que son sobrescritos pueden todavía ser accedidos desde la subclase usando la palabra clave súper.

Coacción

La cadena de conversiones de tipos básicos en Java es:

byte ⇒ short ⇒ int ⇒ long ⇒ float ⇒ double

Un tipo puede ser convertido sin pérdida de información a cualquier otro tipo posterior.

Polimorfismo de inclusión

Métodos genéricos

En muchos lenguajes OO (como Java) existe una superclase común para todas las clases (en este caso Object). Es sencillo crear métodos genéricos que acepten parámetros de cualquier tipo o que devuelvan valores de cualquier tipo usando parámetros o tipos de retorno Object.

Clases envoltorio

Tipo básico	Clase envoltorio
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
boolean	Boolean
char	Character

@informaticapau

Problemas del polimorfismo de inclusión y las colecciones genéricas

- No se pueden hacer colecciones de tipos básicos.
  - Por motivos de eficiencia los tipos básicos de Java (int, float, etc.) no son objetos.
  - Las colecciones basadas en el polimorfismo de inclusión pueden almacenar cualquier subclase de Object, pero no pueden almacenar tipos básicos.
- Los objetos pierden su identidad al entrar en la colección.
  - Todo objeto de una subclase puede ser tratado como un objeto de su superclase, pero mientras es tratado de esta forma no puede acceder a las características concretas de la subclase.
  - El objeto sigue siendo un objeto de la subclase, pero está siendo tratado como un objeto de la superclase.
  - Para poder acceder a las características de la subclase es necesario hacer una conversión explicita de tipos (*type cast*) anteponiendo el nombre de la clase entre paréntesis.
  - El compilador acepta como válidas todas las conversiones explicitas de tipos, pero si la conversión es incorrecta se generará una excepción en tiempo de ejecución.
- No se pueden hacer colecciones de tipos seguras.
  - Si las colecciones son de tipo Object quiere decir que cualquier objeto puede ir en las mismas.

Polimorfismo paramétrico

Parámetros de tipo

- Son parámetros genéricos que especifican un tipo indeterminado.
- T es un identificador que representa a un tipo genérico.
- Podría usarse cualquier identificador para dicho tipo genérico, pero las normas de notación de Java indican que dichos tipos genéricos se representen con una única letra T que representa a un tipo cualquiera.
- También se usan E (*Element*), K (*Key*) o V (*Value*).
- La clase T no se define nunca, será sustituida por el parámetro pasado al hacer el new de la clase.

Instanciar clases genéricas (Java 5)

```
ClaseGenérica<TipoGenérico> nombre = new ClaseGenérica<TipoGenérico>( );
```

Operador diamante (Java 7)

```
ClaseGenérica<TipoGenérico> nombre = new ClaseGenérica<>( );
```

Inferencia de tipos en variables locales (Java 10)

```
var nombre = new ClaseGenérica<TipoGenérico>( );
```

Comodines

- Comodín sin vínculo <?>
  - Representa a cualquier clase.
- Comodín con vínculo superior <? extends Clase >
  - Representa a la clase Clase y a cualquiera de sus subclases.
- Comodín con vínculo inferior <? super Clase>
  - Representa a la clase Clase y a cualquiera de sus superclases.

@informaticapau

El comodín sin vínculo y el comodín con vínculo superior tratan a las colecciones como si fueran de sólo lectura:

- En el primer caso sabemos que la colección es de tipo Object o de alguna de sus subclases.
- En el segundo caso sabemos que la colección es de tipo Clase o de alguna de sus subclases.
- Insertar un elemento en cualquiera de las colecciones puede significar corromper dicha colección.

Principio Get y Put (Productor-Consumidor)

- Utilizar el comodín extends cuando sólo se quiera leer valores de una colección.
- Utilizar el comodín super cuando sólo se quiera escribir valores en una colección.
- No utilizar comodines cuando se quiera leer y escribir al mismo tiempo valores en una colección.

TIPIFICACIÓN

Java tiene un Tipado estático y fuerte.

Restricciones del Tipado

- **Restricción de la declaración**: Todas las entidades del lenguaje (variables, objetos, atributos, parámetros) deben tener un tipo declarado.
    - Existen ocasiones en las que no es necesario declarar los tipos, que son inferidos por el compilador.
- **Restricción de compatibilidad**: En cada asignación *foo* = *bar*, y en cada llamada a una rutina en que se usa *bar* como argumento real para el argumento formal *foo*, el tipo de la fuente *bar* debe ser compatible con el tipo de destino *foo*.
    - La definición de compatibilidad en lenguajes orientados a objetos se basa fundamentalmente en la herencia y en el polimorfismo de inclusión: *bar* es compatible con *foo* si es descendiente de *foo*.
    - Restricción de llamada a la característica: Para poder llamar a una característica (atributo o un método) *foo* desde la clase *bar*, *foo* tiene que estar definida en *bar* o en uno de sus antecesores.

UML

Representación de clases

Modificadores de clases en UML

Modificador Java	Representación UML
public	Generalmente se recoge en la especificación, pero no se representa (podría anteponerse un "+" al nombre de la clase).
abstract	Se pone el nombre en *cursiva*.
final	Restricción {leaf} o propiedad isLeaf

Atributos UML

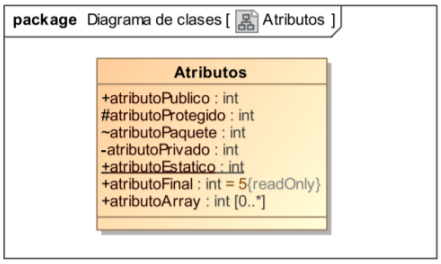
Visibilidad Java	Representación UML
public	+
protected	#
[package]	~
private	-

@informaticapau

Modificador atributo Java	Representación UML
static	Se subraya el atributo
final	Restricción {readOnly}. Si es un blank final puede usarse {frozen}

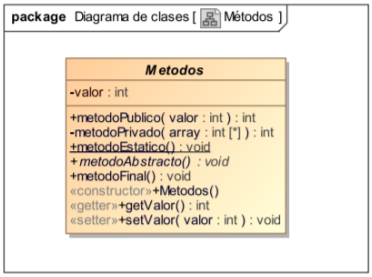
Ejemplo Atributos

```
public class Atributos {
    public int atributoPublico;
    protected int atributoProtegido;
    int atributoPaquete;
    private int atributoPrivado;
    //
    public static int atributoEstatico;
    public final int atributoFinal = 5;
    //
    public int[] atributoArray;
}
```



Métodos UML

Visibilidad: Igual que en los atributos.	
Modificador método Java	**Representación UML**
static	Se subraya el método
abstract	Se pone en *cursiva* el método
final	Restricción {leaf} o propiedad isLeaf

Ejemplo Métodos

```
public abstract class Metodos {
    private int valor;

    public int metodoPublico(int valor) { ... }
    private int metodoPrivado(int[] array) {...}
    public static void metodoEstatico() {}
    public abstract void metodoAbstracto();
    public final void metodoFinal() {};
    public Metodos() {}
    public int getValor() { ... }
    public void setValor(int valor) { ... }
}
```